

**IMPLEMENTASI PARALLEL MERGE SORT MENGGUNAKAN MPI UNTUK
ANALISIS PENGGUNAAN AI DAN DAMPAKNYA TERHADAP KENAIKAN
IPK MAHASISWA**

Dosen Pengampu: Heri Wijayanto, ST.,MT., Ph.D.



OLEH KELOMPOK 2 :

NI WAYAN EKA APRILIANTI	F1D02410021
MUSFIQOH RIZKIA AULIA	F1D02410083
QOMARI AULIYAH	F1D02410086
ANNISA MAKARIMA AKHLAK	F1D02410106

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MATARAM**

2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan *Artificial Intelligence* (AI) dalam beberapa tahun terakhir telah memberikan dampak signifikan dalam berbagai bidang, termasuk pendidikan. AI memungkinkan mahasiswa untuk mengakses informasi dengan cepat, membantu memahami materi, serta menyelesaikan tugas akademik secara lebih efisien. AI merupakan teknologi yang mampu meniru kecerdasan manusia dalam proses pembelajaran, penalaran, dan pemecahan masalah (Mutaqin et al., 2023). Dalam konteks pendidikan, pemanfaatan AI dapat meningkatkan efektivitas pembelajaran serta mendukung proses belajar yang lebih interaktif dan adaptif (Abdullah et al., 2024). Namun demikian, penggunaan AI juga menimbulkan pertanyaan mengenai pengaruhnya terhadap hasil akademik mahasiswa, khususnya Indeks Prestasi Kumulatif (IPK). Beberapa penelitian menunjukkan bahwa penggunaan teknologi AI dan literasi digital memiliki hubungan dengan hasil belajar mahasiswa, baik secara positif maupun negatif tergantung pada cara penggunaannya (Supriyadi & Nasution, 2024). Selain itu, perkembangan AI juga memberikan dampak yang signifikan terhadap sistem pendidikan di Indonesia, baik dari segi kemudahan akses informasi maupun perubahan pola belajar mahasiswa (Waita et al., 2025).

Dalam proses analisis data, pengurutan (*sorting*) merupakan tahap penting untuk menyusun data agar lebih terstruktur dan mudah dianalisis. Salah satu algoritma yang banyak digunakan adalah Merge Sort, yang bekerja berdasarkan konsep *divide and conquer*, yaitu membagi data menjadi bagian kecil, mengurutkannya, kemudian menggabungkannya kembali hingga diperoleh data yang terurut. Algoritma ini memiliki kompleksitas waktu $O(n \log n)$ yang stabil sehingga cocok digunakan untuk pengolahan data dalam jumlah besar. Namun, pengolahan data dalam jumlah besar secara sekuensial masih memiliki keterbatasan dalam hal waktu eksekusi. Oleh karena itu, diperlukan pendekatan pemrosesan paralel yang memungkinkan proses komputasi dilakukan secara bersamaan untuk meningkatkan efisiensi pengolahan data.

Berdasarkan uraian tersebut, penelitian ini bertujuan untuk menganalisis pengaruh penggunaan AI terhadap perubahan IPK mahasiswa dengan memanfaatkan algoritma Merge Sort berbasis pemrosesan paralel. Penelitian ini diharapkan dapat

memberikan manfaat dalam memahami efektivitas penggunaan AI dalam meningkatkan performa akademik mahasiswa serta menunjukkan keunggulan pemrosesan paralel dalam pengolahan data berukuran besar. Selain itu, hipotesis yang diajukan dalam penelitian ini adalah bahwa penggunaan AI memiliki pengaruh terhadap peningkatan IPK mahasiswa.

1.2 Tujuan

Adapun yang menjadi tujuan dalam penelitian ini sebagai berikut:

1. Menerapkan algoritma Merge Sort untuk melakukan proses pengurutan data real-estate. sebagai metode pengurutan pada data *real-estate* yang terdiri dari berbagai atribut seperti harga properti, luas bangunan, dan tingkat kualitas. Algoritma ini dipilih karena memiliki karakteristik stabil, efisien, serta cocok digunakan untuk mengolah data dalam jumlah besar.
2. Menerapkan pemrosesan paralel menggunakan *Message Passing Interface* (MPI). Pendekatan ini dilakukan untuk membagi beban kerja ke dalam beberapa proses yang berjalan secara bersamaan, sehingga dapat mempercepat proses pengurutan data. Dengan membandingkan waktu eksekusi antara proses paralel pada setiap *rank*, efisiensi pemrosesan dapat dianalisis dan dievaluasi.
3. Menganalisis performa algoritma Merge Sort dalam konteks paralel. Hal ini mencakup pengukuran waktu eksekusi pada masing-masing proses, serta proses penggabungan hasil sorting dari setiap proses menjadi satu keluaran yang utuh, sehingga efektivitas algoritma dalam menangani data berskala besar melalui paralelisasi dapat diketahui secara lebih mendalam.
4. Mengevaluasi hasil akhir pengurutan data, untuk memastikan bahwa metode yang diterapkan dapat menghasilkan data yang terstruktur dan siap digunakan untuk analisis lanjutan dalam konteks pengambilan keputusan pada bidang real-estate. Dengan adanya proses penggabungan skor dari berbagai atribut dan penerapan metode sorting yang efisien, data yang dihasilkan, diharapkan dapat digunakan dalam mendukung proses pengambilan keputusan dalam bidang real-estate.

BAB II

DASAR TEORI

2.1 Pemrosesan Paralel

Pemrosesan paralel atau komputasi paralel adalah suatu pendekatan pemecahan masalah yang menggunakan lebih dari satu unit pemrosesan (seperti CPU) untuk mengeksekusi instruksi secara bersamaan atau simultan (Rahmatullah et al., 2021). Operasi ini bekerja dengan melakukan pembagian beban tugas ke beberapa unit pemrosesan, sehingga tugas dapat dipecah dan dieksekusi secara serentak untuk mempercepat waktu total eksekusi komputasi (Subiyakto et al., 2026).

Perbedaan utama antara pemrosesan paralel dan komputasi sekuensial terletak pada metode eksekusi instruksinya. Dalam komputasi sekuensial (serial), prosesor memproses suatu masalah dengan mengeksekusi instruksi satu demi satu secara berurutan (Rahmatullah et al., 2021). Sedangkan, pemrosesan paralel memanfaatkan beberapa prosesor secara bersamaan untuk membagi beban kerja dan mengeksekusi program pada waktu yang serentak (Dhani & Pinandito, 2025).

2.2 *Message Passing Interface (MPI)*

Message Passing Interface (MPI) adalah sebuah antarmuka atau standar protokol pemrograman yang digunakan untuk mengatur alur komunikasi bertukar pesan (*messaging*) antar-proses dalam sistem komputasi paralel (Rahmatullah et al., 2021). Model *message-passing* ini umumnya diterapkan pada arsitektur komputasi memori terdistribusi (*distributed memory*), di mana masing-masing prosesor atau *node* memiliki memori lokalnya sendiri yang terpisah secara fisik. Karena memori tidak dipakai bersama, prosesor-prosesor tersebut harus berkomunikasi dan bertukar data melalui pengiriman pesan (transfer data) agar dapat bekerja sama dalam menyelesaikan sebuah masalah komputasi secara bersamaan (Wibawa et al., 2018).

Fungsi komunikasi dalam MPI terdiri dari komunikasi titik-ke-titik (*point-to-point*) yang terjadi antara dua proses yang spesifik, serta komunikasi kolektif (*collective communication*) yang menyiarkan atau mengumpulkan data dari/ke seluruh prosesor dalam suatu grup (Yang, 2022). Untuk mendukung operasi pembagian dan pengumpulan data secara paralel, MPI menggunakan dua fungsi komunikasi kolektif yang sangat penting, yaitu:

- *Scatter* (Distribusi Data): *Scatter* adalah operasi di mana proses *master* (akar) memecah sebuah kumpulan data besar menjadi beberapa potongan (*chunks*) data

yang proporsional dan terdistribusi secara merata, kemudian mengirimkan (*scatter*) masing-masing potongan tersebut ke semua proses *worker* yang ada dalam kelompok komputasi. Melalui metode ini, masing-masing *worker* mendapatkan sub-data independen untuk diproses tanpa saling mengganggu (misalnya: untuk diurutkan secara lokal menggunakan algoritma pengurutan) (Yang, 2022).

- *Gather* (Pengumpulan Data): *Gather* adalah kebalikan dari fungsi *scatter*. Setelah para proses *worker* menyelesaikan tugas komputasi dan pengurutan pada potongan datanya masing-masing, *master* akan memanggil fungsi *gather* untuk menarik, mengumpulkan, dan menyatukan kembali seluruh data hasil pekerjaan dari para *worker* tersebut ke dalam satu lokasi di *node master* (Yang, 2022).

2.3 Merge Sort

Algoritma Merge Sort adalah salah satu algoritma pengurutan klasik yang menggunakan pendekatan *divide-and-conquer* (pecah dan taklukkan). Pendekatan ini bekerja dengan memecah masalah komputasi besar secara rekursif menjadi sub-masalah yang lebih kecil hingga menjadi sangat sederhana untuk diselesaikan secara langsung, dan kemudian hasilnya digabungkan kembali (Yang, 2022).

Secara spesifik, langkah-langkah dasar algoritma Merge Sort adalah sebagai berikut:

1. Memeriksa apakah sekumpulan data (*array*) memiliki kurang dari dua elemen; jika ya, proses dihentikan karena dianggap sudah terurut.
2. Membagi kumpulan data yang belum terurut menjadi dua bagian yang sama besar, yaitu bagian kiri dan kanan.
3. Mengurutkan kedua bagian tersebut secara terpisah (lokal).
4. Menggabungkan (*merge*) kembali kedua bagian yang sudah terurut tersebut menjadi satu kumpulan data utuh yang berurutan.

Keunggulan utama dari algoritma Merge Sort adalah tingkat kestabilan performanya. Algoritma ini memiliki kompleksitas waktu $O(n \log n)$, baik untuk kondisi rata-rata (*average case*), kasus terbaik (*best case*), maupun kasus terburuk (*worst case*). Ini berarti bahwa kecepatan pengurutannya tidak akan banyak terpengaruh oleh bagaimana kondisi awal data sebelum diurutkan (baik itu sudah hampir terurut, teracak sempurna, atau terurut terbalik). Meski stabil secara waktu, algoritma ini membutuhkan ruang memori tambahan sebesar $O(n)$ untuk menampung data sementara saat proses penggabungan (Yang, 2022).

BAB III

METODOLOGI

Penelitian ini menerapkan metode algoritma *Merge Sort* secara paralel menggunakan pustaka *mpi4py* berbasis Python. Tujuan penelitian adalah untuk mengukur efisiensi waktu eksekusi dalam proses pengurutan data real-estate berdasarkan skor agregat dari harga, luas properti, dan kualitas bangunan. Metodologi dibagi ke dalam beberapa tahapan, yaitu: pemilihan dataset, proses normalisasi, implementasi algoritma paralel, dan analisis hasil.

1. Pemilihan Dataset dan Proses Normalisasi

Dataset yang digunakan dalam penelitian ini adalah *Dubai Properties Dataset* berisi sekitar satu juta entri dengan atribut harga, ukuran, dan kualitas. Dataset diunduh dalam format CSV dan dibaca menggunakan *pandas*. Tidak ditemukan data kosong sehingga tidak diperlukan proses imputasi. Selanjutnya, atribut *quality* yang bersifat kategorikal dikonversi ke bentuk numerik dengan pemetaan, Low=1, Medium=2, High=3. Tiga skor dihitung menggunakan normalisasi di masing-masing rank (lokal), menggunakan fungsi *min_max_normalize* pada atribut *price*, *size_in_sqft*, dan *quality_numeric*, sebelum dihitung rata-rata ke *total_score*.

2. Distribusi Data dan Implementasi Paralel

Sistem paralel dibangun menggunakan pustaka *mpi4py*, yang merupakan implementasi protokol Message Passing Interface (MPI) dalam bahasa pemrograman Python. Proses utama (rank 0) membaca keseluruhan dataset, membaginya ke sejumlah bagian sesuai jumlah proses (rank), lalu mengirimkan bagian data ke masing-masing proses dengan *comm.send()*. Setiap proses, termasuk rank 0, akan melakukan pemrosesan pada data masing-masing. Komunikasi antar-proses dilakukan menggunakan metode *comm.send()* dan *comm.recv()*.

3. Proses Pengurutan Lokal

Setiap proses melakukan penghitungan *total_score* seperti dijelaskan di tahap 1, kemudian menjalankan algoritma Merge-Sort secara lokal pada bagian data tersebut. Merge-Sort diimplementasikan secara manual dalam bentuk fungsi rekursif *manual_mergesort()* dan fungsi *manual_merge()* untuk proses penggabungan dua list. Proses ini dilakukan pada struktur data berbentuk list of dictionary (records dari DataFrame). Waktu eksekusi setiap proses diukur

menggunakan `time.time()`.

4. Penggabungan Hasil

Setelah proses pengurutan selesai, setiap proses mengirimkan hasil data terurut beserta waktu pemrosesannya ke proses utama (rank 0). Proses utama kemudian

menjalankan proses parallel merge tree secara bertahap untuk menggabungkan semua list terurut menjadi satu list akhir yang sudah sepenuhnya terurut berdasarkan total_score. Proses ini menggunakan pendekatan pairwise merging yang dilakukan bertahap dalam loop hingga hanya tersisa satu list.

5. Analisis Kerja

Analisis kerja dilakukan oleh proses utama (rank 0) dengan menghitung total waktu eksekusi menggunakan `time.time()` serta menyimpan waktu sorting dari masing-masing proses dalam file CSV (`waktu_sorting.csv`). Hasil pengurutan akhir disimpan dalam file `final_sorted_output.csv`. Analisis dilakukan dengan membandingkan waktu masing-masing proses untuk menilai efisiensi distribusi beban. Hasil ini dapat divisualisasikan lebih lanjut menggunakan `matplotlib` atau `seaborn`.

BAB IV

ISI DAN PEMBAHASAN

4.1 Analisis Data

```
from mpi4py import MPI
import csv
from collections import defaultdict
import math
```

Script “from mpi4py import MPI”, “import csv”, “from collections import defaultdict”, dan “import math” merupakan bagian awal program yang berfungsi untuk memanggil beberapa *library* yang dibutuhkan agar program dapat berjalan dengan baik. *Library* mpi4py digunakan untuk mengaktifkan mekanisme pemrosesan paralel menggunakan konsep Message Passing Interface (MPI), sehingga program dapat dijalankan oleh beberapa proses secara bersamaan dan memungkinkan komunikasi antar proses, seperti pembagian data dan pengumpulan hasil perhitungan. Selanjutnya, *library* csv digunakan untuk membaca file *dataset* yang berformat CSV, sehingga data di dalam file dapat diakses dengan mudah berdasarkan nama kolom. *Library* defaultdict dari modul *collections* digunakan untuk mempermudah proses penyimpanan dan perhitungan data dalam bentuk *dictionary*, karena struktur ini secara otomatis memberikan nilai awal ketika sebuah *key* baru digunakan, sehingga proses penjumlahan data menjadi lebih praktis dan efisien. Sementara itu, *library* math digunakan untuk menyediakan fungsi-fungsi matematika, khususnya fungsi pembulatan ke atas (ceil), yang berguna dalam menentukan ukuran pembagian data agar dapat didistribusikan secara merata ke setiap proses dalam sistem paralel.

```
COMM = MPI.COMM_WORLD
RANK = COMM.Get_rank()
SIZE = COMM.Get_size()

DATA_FILE = "AI_Impact_Student_Life_2026.csv"
```

Script COMM = MPI.COMM_WORLD, RANK = COMM.Get_rank(), dan SIZE = COMM.Get_size() merupakan bagian inisialisasi dasar dalam pemrograman paralel menggunakan MPI. Variabel COMM digunakan untuk menyimpan objek komunikasi utama (MPI.COMM_WORLD), yaitu kumpulan seluruh proses yang terlibat dalam eksekusi program. Selanjutnya, RANK = COMM.Get_rank() digunakan untuk mengetahui nomor identitas dari setiap proses yang sedang berjalan, sehingga setiap proses dapat dibedakan satu sama lain dan memiliki tugas yang berbeda, misalnya proses dengan rank 0 biasanya bertindak sebagai proses

utama (root). Sementara itu, `SIZE = COMM.Get_size()` digunakan untuk mengetahui jumlah total proses yang berjalan dalam sistem MPI, sehingga program dapat menentukan bagaimana data akan dibagi secara merata ke setiap proses. Pada baris `DATA_FILE = "AI_Impact_Student_Life_2026.csv"`, ditentukan nama *file* dataset yang akan digunakan sebagai sumber data dalam program, sehingga *file* tersebut dapat dipanggil kembali saat proses pembacaan data dilakukan.

```
def merge(left, right, key_index):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i][key_index] >= right[j][key_index]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
def merge_sort(arr, key_index):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid], key_index)
    right = merge_sort(arr[mid:], key_index)
    return merge(left, right, key_index)
```

Fungsi `merge()` dan `merge_sort()` merupakan implementasi dari algoritma Merge Sort yang digunakan untuk mengurutkan data berdasarkan nilai tertentu. Fungsi `merge()` bertugas untuk menggabungkan dua buah list yang sudah terurut, yaitu `left` dan `right`, menjadi satu list baru yang tetap dalam keadaan terurut. Pada awal fungsi, dibuat list kosong bernama `result` untuk menyimpan hasil penggabungan, serta dua variabel indeks `i` dan `j` yang digunakan untuk menelusuri elemen pada masing-masing list. Proses penggabungan dilakukan menggunakan perulangan `while` yang membandingkan elemen dari kedua list berdasarkan indeks tertentu (`key_index`). Jika nilai pada list kiri lebih besar atau sama dengan nilai pada list kanan, maka elemen dari list kiri dimasukkan ke dalam `result`, kemudian indeks `i` bertambah satu. Jika tidak, maka elemen dari list kanan dimasukkan dan indeks `j` ditambah satu. Setelah salah satu list habis dibandingkan, sisa elemen dari list lainnya langsung ditambahkan ke dalam `result` menggunakan `extend()`, sehingga seluruh data tetap tersimpan tanpa terlewat. Fungsi ini kemudian mengembalikan list hasil penggabungan yang sudah terurut dalam urutan menurun.

Selanjutnya, fungsi `merge_sort()` digunakan untuk melakukan proses pengurutan utama dengan metode pembagian data secara rekursif. Fungsi ini pertama-tama memeriksa apakah jumlah elemen dalam list `arr` kurang dari atau sama dengan satu. Jika kondisi tersebut terpenuhi, maka list langsung dikembalikan karena sudah dianggap terurut. Jika jumlah elemen lebih dari satu, maka list dibagi menjadi dua bagian menggunakan titik tengah (`mid`). Bagian kiri (`left`) dan kanan (`right`) kemudian diproses kembali menggunakan fungsi `merge_sort()` secara rekursif hingga setiap bagian menjadi sangat kecil. Setelah kedua bagian selesai diurutkan, fungsi `merge()` dipanggil untuk menggabungkan kedua bagian tersebut menjadi satu list yang terurut. Dengan cara ini, seluruh data akan terurut secara bertahap hingga menghasilkan urutan akhir yang sesuai dengan nilai pada indeks yang ditentukan.

```
def load_csv(filename):
    records = []
    with open(filename, newline="", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        for row in reader:
            ai_tool = row["Primary_AI_Tool"]
            gpa_before = float(row["GPA_Baseline"])
            gpa_after = float(row["GPA_Post_AI"])
            delta = gpa_after - gpa_before
            records.append((ai_tool, delta))
    return records
```

Fungsi `load_csv(filename)` digunakan untuk membaca data dari file CSV dan menyiapkannya dalam bentuk yang dapat diproses oleh program. Pada awal fungsi, dibuat list kosong bernama `records` yang berfungsi untuk menyimpan hasil pembacaan data. Selanjutnya, file dibuka menggunakan perintah `with open()` dengan parameter `newline=""` dan `encoding="utf-8"` agar file dapat dibaca dengan benar tanpa masalah format baris atau karakter. Data di dalam file kemudian dibaca menggunakan `csv.DictReader`, yang memungkinkan setiap baris data diakses berdasarkan nama kolom. Di dalam perulangan `for`, setiap baris data diambil dan nilai pada kolom `"Primary_AI_Tool"` disimpan ke variabel `ai_tool`. Kemudian nilai pada kolom `"GPA_Baseline"` dan `"GPA_Post_AI"` diambil dan diubah menjadi tipe data `float` agar dapat dilakukan perhitungan numerik. Setelah itu, dihitung selisih nilai GPA (`delta`) dengan cara mengurangkan GPA sebelum menggunakan AI dari GPA setelah menggunakan AI. Nilai `delta` ini menunjukkan perubahan GPA yang terjadi setelah penggunaan AI. Setiap pasangan data berupa `(ai_tool, delta)` kemudian disimpan ke dalam list `records`. Setelah seluruh data selesai dibaca dan diproses, fungsi mengembalikan list `records` yang berisi seluruh data yang telah

disiapkan untuk tahap pengolahan selanjutnya.

```
def main():
    start_time = MPI.Wtime()

    print(f"Rank {RANK} aktif dari {SIZE}")
    if RANK == 0:
        all_data = load_csv(DATA_FILE)
        all_data = all_data * 50000
        print(f"Total data: {len(all_data)}")
        chunk_size = math.ceil(len(all_data) / SIZE)
        chunks = []
        for i in range(SIZE):
            start = i * chunk_size
            end = start + chunk_size

            chunks.append(all_data[start:end])
    else:
        chunks = None
```

Fungsi `main()` pada bagian ini berperan sebagai awal dari proses utama program, khususnya dalam menyiapkan data yang akan diproses secara paralel. Pada baris `start_time = MPI.Wtime()`, program mulai mencatat waktu awal eksekusi yang nantinya digunakan untuk menghitung total waktu yang dibutuhkan selama program berjalan. Selanjutnya, perintah `print(f"Rank {RANK} aktif dari {SIZE}")` digunakan untuk menampilkan informasi bahwa setiap proses MPI telah aktif, sekaligus menunjukkan nomor proses (`RANK`) dan jumlah total proses (`SIZE`) yang digunakan dalam eksekusi paralel.

Pada bagian `if RANK == 0:`, hanya proses dengan rank 0 yang bertugas membaca data dari file menggunakan fungsi `load_csv(DATA_FILE)`. Hal ini dilakukan untuk menghindari pembacaan file yang berulang oleh banyak proses sekaligus, sehingga lebih efisien. Setelah data berhasil dibaca, terdapat baris `all_data = all_data * 50000`, yang berfungsi untuk menggandakan data sebanyak 50.000 kali. Langkah ini biasanya dilakukan untuk memperbesar jumlah data sehingga pengujian performa paralel menjadi lebih terlihat, terutama saat mengukur waktu eksekusi program dengan jumlah data yang besar. Kemudian, program menampilkan jumlah total data menggunakan `print(f"Total data: {len(all_data)}")`.

Selanjutnya, ukuran pembagian data (`chunk_size`) dihitung menggunakan `math.ceil(len(all_data) / SIZE)`, yang bertujuan untuk menentukan berapa banyak data yang akan diberikan ke setiap proses secara merata. Penggunaan `math.ceil()` memastikan bahwa seluruh data tetap terbagi meskipun jumlah data tidak habis dibagi dengan jumlah proses. Setelah itu, dibuat list kosong bernama `chunks` yang akan menyimpan bagian-bagian data yang telah dibagi. Melalui perulangan `for`,

setiap bagian data ditentukan dengan menghitung indeks awal (start) dan indeks akhir (end), kemudian potongan data tersebut dimasukkan ke dalam list chunks. Sementara itu, pada bagian else:, proses selain rank 0 akan mengisi variabel chunks dengan nilai None, karena hanya proses utama yang bertugas menyiapkan dan membagi data sebelum didistribusikan ke proses lain.

```
local_data = COMM.scatter(chunks, root=0)
print(f"Rank {RANK} menerima {len(local_data)} data")
```

Baris `local_data = COMM.scatter(chunks, root=0)` digunakan untuk mendistribusikan data yang telah dibagi sebelumnya ke seluruh proses yang terlibat dalam sistem MPI. Pada tahap ini, proses dengan `RANK == 0` bertindak sebagai root yang mengirimkan setiap bagian data (chunks) ke proses lain, sehingga setiap proses menerima sebagian data yang berbeda. Metode scatter memastikan bahwa data dibagi secara merata sesuai jumlah proses (SIZE), sehingga setiap proses dapat bekerja secara paralel pada data yang telah diterimanya. Hasil pembagian tersebut disimpan dalam variabel `local_data`, yang berisi data lokal milik masing-masing proses untuk diproses lebih lanjut.

Selanjutnya, baris `print(f"Rank {RANK} menerima {len(local_data)} data")` digunakan untuk menampilkan jumlah data yang diterima oleh setiap proses. Informasi ini berguna untuk memastikan bahwa proses distribusi data berjalan dengan benar dan setiap proses telah menerima bagian data sesuai dengan yang diharapkan. Dengan adanya pembagian data menggunakan scatter, proses pengolahan data dapat dilakukan secara paralel, sehingga waktu eksekusi program dapat menjadi lebih efisien dibandingkan pemrosesan secara berurutan (sequential).

```
local_count = defaultdict(int)
local_delta_sum = defaultdict(float)
for ai_tool, delta in local_data:
    local_count[ai_tool] += 1
    local_delta_sum[ai_tool] += delta
```

Pada bagian ini dilakukan proses perhitungan lokal pada setiap proses MPI berdasarkan data yang telah diterima. Baris `local_count = defaultdict(int)` digunakan untuk membuat dictionary yang berfungsi menghitung jumlah penggunaan setiap jenis AI, dengan nilai awal berupa bilangan bulat nol. Sementara itu, baris `local_delta_sum = defaultdict(float)` digunakan untuk membuat dictionary lain yang berfungsi menjumlahkan total perubahan GPA (delta) untuk setiap jenis AI, dengan nilai awal berupa bilangan desimal nol. Selanjutnya, perulangan for

ai_tool, delta in local_data: digunakan untuk menelusuri setiap data lokal yang diterima oleh proses. Pada setiap iterasi, nilai ai_tool menunjukkan nama AI yang digunakan, sedangkan delta menunjukkan perubahan GPA yang terjadi. Baris local_count[ai_tool] += 1 berfungsi menambahkan jumlah kemunculan AI tersebut, sehingga dapat diketahui seberapa sering AI digunakan dalam data lokal. Kemudian, baris local_delta_sum[ai_tool] += delta digunakan untuk menambahkan nilai perubahan GPA ke total delta yang sesuai dengan AI tersebut. Dengan cara ini, setiap proses MPI melakukan perhitungan secara mandiri terhadap bagian datanya masing-masing, sehingga mempercepat proses pengolahan secara keseluruhan sebelum hasilnya dikirim kembali ke proses utama.

```
all_counts = COMM.gather(local_count, root=0)
all_deltas = COMM.gather(local_delta_sum, root=0)
```

Baris all_counts = COMM.gather(local_count, root=0) dan all_deltas = COMM.gather(local_delta_sum, root=0) digunakan untuk mengumpulkan hasil perhitungan lokal dari setiap proses kembali ke proses utama (root). Metode gather berfungsi sebagai kebalikan dari scatter, yaitu mengirimkan data dari semua proses ke satu proses tertentu, dalam hal ini proses dengan RANK == 0. Pada baris pertama, setiap proses mengirimkan dictionary local_count yang berisi jumlah penggunaan masing-masing AI, lalu seluruh data tersebut dikumpulkan dalam variabel all_counts pada proses root dalam bentuk list yang berisi dictionary dari setiap proses. Pada baris kedua, proses yang sama dilakukan untuk dictionary local_delta_sum, yaitu mengirimkan total perubahan GPA dari setiap proses dan mengumpulkannya dalam variabel all_deltas. Dengan adanya proses gather ini, seluruh hasil perhitungan yang sebelumnya dilakukan secara paralel dapat digabungkan kembali di proses utama untuk dilakukan pengolahan lanjutan, seperti penggabungan total data dan perhitungan rata-rata perubahan GPA untuk setiap AI.

```
if RANK == 0:

    total_count = defaultdict(int)
    total_delta = defaultdict(float)
```

Bagian if RANK == 0: menunjukkan bahwa langkah selanjutnya hanya dijalankan oleh proses utama (root), yaitu proses dengan nomor rank 0. Hal ini dilakukan karena setelah proses gather, seluruh hasil perhitungan dari setiap proses MPI sudah terkumpul di root, sehingga hanya root yang perlu melakukan pengolahan lanjutan. Pada baris total_count = defaultdict(int), dibuat sebuah

dictionary yang berfungsi untuk menyimpan jumlah total penggunaan setiap jenis AI dari seluruh proses, dengan nilai awal berupa nol untuk setiap key baru. Selanjutnya, baris `total_delta = defaultdict(float)` digunakan untuk membuat dictionary lain yang berfungsi menyimpan total akumulasi perubahan GPA (delta) untuk setiap AI dari seluruh data yang telah dikumpulkan. Kedua dictionary ini nantinya akan digunakan untuk menggabungkan hasil dari semua proses dan menjadi dasar dalam menghitung rata-rata perubahan GPA pada tahap berikutnya.

```
for d in all_counts:
    for tool in d:
        total_count[tool] += d[tool]
for d in all_deltas:
    for tool in d:
        total_delta[tool] += d[tool]
```

Pada bagian ini dilakukan proses penggabungan hasil perhitungan dari seluruh proses MPI ke dalam satu data total pada proses root. Perulangan `for d in all_counts:` digunakan untuk menelusuri setiap dictionary yang terdapat dalam list `all_counts`, di mana setiap dictionary tersebut merupakan hasil perhitungan jumlah penggunaan AI dari masing-masing proses. Selanjutnya, perulangan `for tool in d:` digunakan untuk menelusuri setiap jenis AI yang ada di dalam dictionary tersebut. Baris `total_count[tool] += d[tool]` berfungsi untuk menambahkan jumlah penggunaan AI dari setiap proses ke dalam dictionary `total_count`, sehingga pada akhirnya diperoleh jumlah total penggunaan setiap AI dari seluruh data. Proses yang sama dilakukan pada bagian `for d in all_deltas:` yang bertujuan untuk menggabungkan total perubahan GPA dari setiap proses. Pada perulangan ini, setiap nilai perubahan GPA yang tersimpan dalam `all_deltas` dijumlahkan dan disimpan dalam dictionary `total_delta` menggunakan baris `total_delta[tool] += d[tool]`. Dengan cara ini, seluruh hasil perhitungan lokal dari masing-masing proses berhasil digabungkan menjadi satu data global yang mewakili keseluruhan dataset.

```
usage_list = list(total_count.items())

avg_delta = {}

for tool in total_count:

    avg = total_delta[tool] / total_count[tool]
    avg_delta[tool] = avg

impact_list = list(avg_delta.items())
```

Pada bagian ini dilakukan proses persiapan data sebelum dilakukan pengurutan. Baris `usage_list = list(total_count.items())` digunakan untuk mengubah dictionary `total_count` menjadi bentuk list yang berisi pasangan data (`tool`,

jumlah_penggunaan). Perubahan bentuk ini diperlukan karena proses pengurutan menggunakan algoritma Merge Sort lebih mudah dilakukan pada struktur data list. Selanjutnya, dibuat dictionary kosong bernama avg_delta yang berfungsi untuk menyimpan nilai rata-rata perubahan GPA untuk setiap jenis AI. Perulangan for tool in total_count: digunakan untuk menelusuri setiap jenis AI yang telah dihitung sebelumnya. Pada setiap iterasi, nilai rata-rata perubahan GPA dihitung dengan rumus $avg = total_delta[tool] / total_count[tool]$, yaitu dengan membagi total perubahan GPA dengan jumlah penggunaan AI tersebut. Hasil perhitungan rata-rata ini kemudian disimpan ke dalam dictionary avg_delta dengan key berupa nama AI. Setelah seluruh rata-rata berhasil dihitung, baris impact_list = list(avg_delta.items()) digunakan untuk mengubah dictionary avg_delta menjadi bentuk list yang berisi pasangan (tool, rata-rata_delta), sehingga data tersebut siap untuk diproses pada tahap pengurutan selanjutnya.

```
sorted_usage = merge_sort(
    usage_list,
    key_index=1
)
sorted_impact = merge_sort(
    impact_list,
    key_index=1
)
```

Pada bagian ini dilakukan proses pengurutan data menggunakan algoritma Merge Sort yang telah dibuat sebelumnya. Baris sorted_usage = merge_sort(usage_list, key_index=1) digunakan untuk mengurutkan data pada usage_list, yaitu list yang berisi pasangan (tool, jumlah_penggunaan). Parameter key_index=1 menunjukkan bahwa proses pengurutan dilakukan berdasarkan nilai pada indeks ke-1, yaitu jumlah penggunaan AI, sehingga AI dengan jumlah penggunaan terbesar akan berada di urutan teratas. Hasil pengurutan tersebut kemudian disimpan dalam variabel sorted_usage. Selanjutnya, baris sorted_impact = merge_sort(impact_list, key_index=1) digunakan untuk mengurutkan data pada impact_list, yaitu list yang berisi pasangan (tool, rata-rata_delta). Sama seperti sebelumnya, parameter key_index=1 digunakan agar pengurutan dilakukan berdasarkan nilai rata-rata perubahan GPA. Hasil pengurutan ini disimpan dalam variabel sorted_impact. Dengan adanya proses ini, data menjadi lebih terstruktur dan memudahkan dalam menampilkan daftar AI yang paling sering digunakan serta AI yang memiliki dampak terbesar terhadap perubahan GPA.

```
print("\n=====")
print("AI PALING SERING DIGUNAKAN")
```



```

print("=====")

for i, (tool, count) in enumerate(sorted_usage, 1):

    print(
        f"{i}. {tool:<20} "
        f"| Jumlah: {count}"
    )

print("\n=====")
print("AI PALING BERDAMPAK KE GPA")
print("=====")

for i, (tool, avg) in enumerate(sorted_impact, 1):

    print(
        f"{i}. {tool:<20} "
        f"| Rata-rata GPA: {avg:.3f}"
    )

    end_time = MPI.Wtime()

if RANK == 0:
    print("\nWaktu    eksekusi:",    end_time    -    start_time,
"detik")

```

Bagian ini digunakan untuk menampilkan hasil akhir pengolahan data yang telah diurutkan. Perintah `print()` pertama menampilkan judul AI PALING SERING DIGUNAKAN, kemudian perulangan `for i, (tool, count) in enumerate(sorted_usage, 1):` digunakan untuk menampilkan daftar AI beserta jumlah penggunaannya secara berurutan, dengan nomor urut yang dimulai dari angka 1. Selanjutnya, bagian AI PALING BERDAMPAK KE GPA menampilkan daftar AI berdasarkan rata-rata perubahan GPA menggunakan perulangan pada `sorted_impact`, di mana nilai rata-rata ditampilkan dengan tiga angka di belakang koma agar lebih rapi. Setelah seluruh hasil ditampilkan, baris `end_time = MPI.Wtime()` mencatat waktu akhir eksekusi. Terakhir, pada kondisi `if RANK == 0:`, waktu total eksekusi dihitung dari selisih waktu akhir dan waktu awal, lalu ditampilkan dalam satuan detik oleh proses utama saja agar tidak terjadi pengulangan output dari proses lain.

```

if __name__ == "__main__":
    main()

```

Bagian `if __name__ == "__main__":` digunakan untuk memastikan bahwa fungsi `main()` hanya dijalankan ketika file program dijalankan secara langsung, bukan ketika file tersebut dipanggil atau diimpor sebagai modul di program lain. Kondisi ini berfungsi sebagai titik awal eksekusi program, sehingga ketika program dijalankan, perintah `main()` akan dipanggil dan seluruh rangkaian proses utama, seperti pembacaan data, pemrosesan paralel, hingga penampilan hasil, dapat

berjalan sesuai urutan yang telah ditentukan.

4.2 Perbandingan Hasil

```
root@ms-server1:~# ./deploympi
mpi_sort.py
data.csv
100% 2944 8.39B/s 00:00
100% 436KB 73.49B/s 00:00
100% 2944 6.09B/s 00:00
100% 436KB 61.59B/s 00:00
100% 2944 6.59B/s 00:00
100% 436KB 67.29B/s 00:00
root@ms-server1:~# mpirun --allow-run-as-root -np 1 python3 mpi_sort.py
/usr/lib/python3/dist-packages/numpy/core/fromnumeric.py:59: FutureWarning: 'DataFrame.swapaxes' is deprecated and will be removed in a future version. Please use 'DataFrame.transpose' instead.
return bound(*args, **kargs)
File akhir dislipan di final.sorted.output.csv
Waktu sorting dislipan di waktu.sorting.csv
Total waktu eksekusi: 0.1555 detik
root@ms-server1:~# mpirun --allow-run-as-root --hostfile hosts.txt -np 2 python3 mpi_sort.py
/usr/lib/python3/dist-packages/numpy/core/fromnumeric.py:59: FutureWarning: 'DataFrame.swapaxes' is deprecated and will be removed in a future version. Please use 'DataFrame.transpose' instead.
return bound(*args, **kargs)
File akhir dislipan di final.sorted.output.csv
Waktu sorting dislipan di waktu.sorting.csv
Total waktu eksekusi: 0.1724 detik
root@ms-server1:~# mpirun --allow-run-as-root --hostfile hosts.txt -np 3 python3 mpi_sort.py
/usr/lib/python3/dist-packages/numpy/core/fromnumeric.py:59: FutureWarning: 'DataFrame.swapaxes' is deprecated and will be removed in a future version. Please use 'DataFrame.transpose' instead.
return bound(*args, **kargs)
File akhir dislipan di final.sorted.output.csv
Waktu sorting dislipan di waktu.sorting.csv
Total waktu eksekusi: 0.1352 detik
```

Gambar 1.1 Percobaan Seribu Data

Pada gambar diatas, dilakukan percobaan untuk 1000 data dengan menggunakan 1, 2 dan 3 proses. Pada skenario pertama, program dijalankan menggunakan proses (-np 1) di satu mesin, dan menghasilkan waktu eksekusi total sebesar 0,1555 detik. Sementara pada skenario kedua, program dijalankan menggunakan (-np 2) proses yang tersebar pada beberapa node, dengan hasil waktu eksekusi yaitu 0,1724 detik. Selanjutnya pada skenario ketiga, program dijalankan menggunakan (-np 3) dengan hasil eksekusi lebih cepat yaitu 0.1352 detik.

```
root@ms-server1:~# ./deploympi
mpi_sort.py
dabai.properties_10k.csv
100% 2960 6.19B/s 00:00
100% 1021KB 98.79B/s 00:00
100% 2960 3.98B/s 00:00
100% 1021KB 111.59B/s 00:00
100% 2960 4.79B/s 00:00
100% 1021KB 111.49B/s 00:00
root@ms-server1:~# mpirun --allow-run-as-root -np 1 python3 mpi_sort.py
/usr/lib/python3/dist-packages/numpy/core/fromnumeric.py:59: FutureWarning: 'DataFrame.swapaxes' is deprecated and will be removed in a future version. Please use 'DataFrame.transpose' instead.
return bound(*args, **kargs)
File akhir dislipan di final.sorted.output.csv
Waktu sorting dislipan di waktu.sorting.csv
Total waktu eksekusi: 0.4595 detik
root@ms-server1:~# mpirun --allow-run-as-root --hostfile hosts.txt -np 2 python3 mpi_sort.py
/usr/lib/python3/dist-packages/numpy/core/fromnumeric.py:59: FutureWarning: 'DataFrame.swapaxes' is deprecated and will be removed in a future version. Please use 'DataFrame.transpose' instead.
return bound(*args, **kargs)
File akhir dislipan di final.sorted.output.csv
Waktu sorting dislipan di waktu.sorting.csv
Total waktu eksekusi: 0.3357 detik
root@ms-server1:~# mpirun --allow-run-as-root --hostfile hosts.txt -np 3 python3 mpi_sort.py
/usr/lib/python3/dist-packages/numpy/core/fromnumeric.py:59: FutureWarning: 'DataFrame.swapaxes' is deprecated and will be removed in a future version. Please use 'DataFrame.transpose' instead.
return bound(*args, **kargs)
File akhir dislipan di final.sorted.output.csv
Waktu sorting dislipan di waktu.sorting.csv
Total waktu eksekusi: 0.2860 detik
```

Gambar 1.2 Percobaan Sepuluh Ribu Data

Pada gambar di atas, dilakukan percobaan untuk 10.000 data. Percobaan pertama dijalankan secara serial menggunakan satu proses (-np 1) yang artinya seluruh proses dilakukan oleh 1 server saja tanpa pembagian proses, waktu eksekusi yang tercatat adalah 0,4595 detik. Pada percobaan kedua program dijalan dengan 2 server secara paralel (-np 2) yang di panggil pada file hosts.txt menunjukkan waktu eksekusi 0,3357 detik. sementara, pada percobaan ketiga, program dijalankan dengan 3server secara paralel (-np 3) menunjukkan waktu eksekusi 0.2860 detik. Ini menunjukkan adanya peningkatan kinerja dan penghematan waktu ketika program di lakukan secara paralel.

```
# root@w-server1:~# ./deploympi
mpi_sort.py
dubai_properties_100k.csv
mpi_sort.py
dubai_properties_100k.csv
mpi_sort.py
dubai_properties_100k.csv
# root@w-server1:~# mpirun --allow-run-as-root -np 1 python3 mpi_sort.py
/usr/lib/python3/dist-packages/numpy/core/fromnumeric.py:59: FutureWarning: 'DataFrame.swapaxes' is deprecated and will be removed in a future version. Please use 'DataFrame.transpose' instead.
return bound(*args, **kargs)
File akhir disalin di final_sorted_output.csv
Waktu sorting disalin di waktu_sorting.csv
Total waktu eksekusi: 4.4911 detik
# root@w-server1:~# mpirun --allow-run-as-root --hostfile hosts.txt -np 2 python3 mpi_sort.py
/usr/lib/python3/dist-packages/numpy/core/fromnumeric.py:59: FutureWarning: 'DataFrame.swapaxes' is deprecated and will be removed in a future version. Please use 'DataFrame.transpose' instead.
return bound(*args, **kargs)
File akhir disalin di final_sorted_output.csv
Waktu sorting disalin di waktu_sorting.csv
Total waktu eksekusi: 3.1599 detik
# root@w-server1:~# mpirun --allow-run-as-root --hostfile hosts.txt -np 3 python3 mpi_sort.py
/usr/lib/python3/dist-packages/numpy/core/fromnumeric.py:59: FutureWarning: 'DataFrame.swapaxes' is deprecated and will be removed in a future version. Please use 'DataFrame.transpose' instead.
return bound(*args, **kargs)
File akhir disalin di final_sorted_output.csv
Waktu sorting disalin di waktu_sorting.csv
Total waktu eksekusi: 2.2970 detik
```

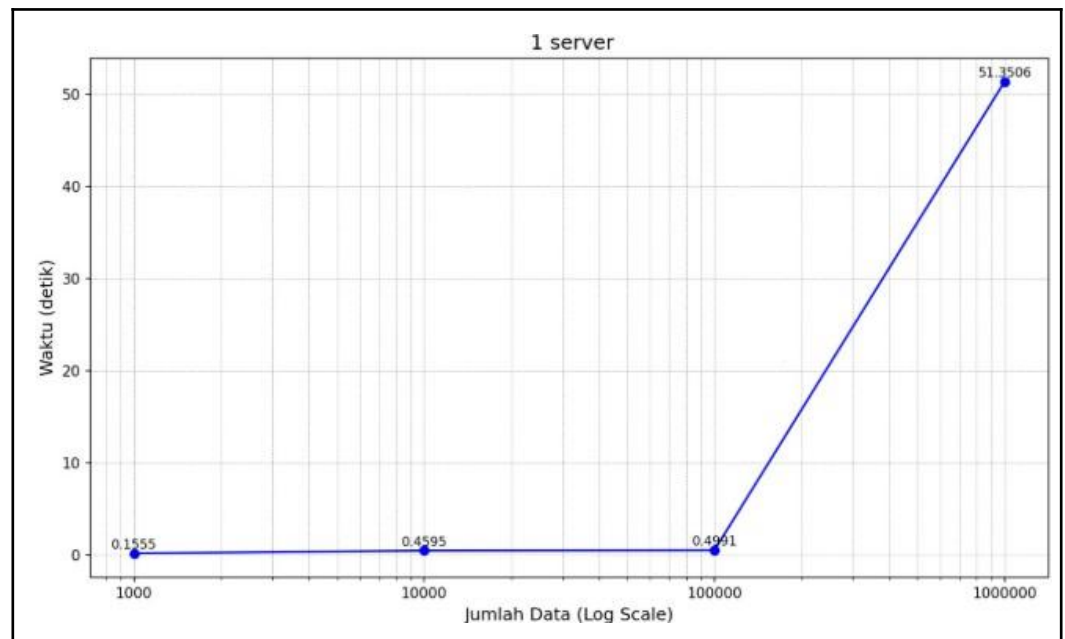
Gambar 1.3 Seratus Ribu Data

Pada gambar diatas, dilakukan percobaan untuk 100.000 data dengan menggunakan 1, 2, dan 3 proses. Pada skenario pertama dengan satu proses (-np 1), program membutuhkan waktu eksekusi sebesar 4,4911 detik. Ketika dijalankan dengan dua proses (-np 2), waktu eksekusi menurun menjadi 3,1599 detik, menunjukkan peningkatan efisiensi pemrosesan akibat pembagian beban kerja. Sementara itu, pada skenario dengan tiga proses (-np 3), waktu eksekusi kembali menurun secara signifikan menjadi 2,2970 detik.

```
# root@w-server1:~# ./deploympi
mpi_sort.py
dubai_properties_1m.csv
mpi_sort.py
dubai_properties_1m.csv
mpi_sort.py
dubai_properties_1m.csv
# root@w-server1:~# mpirun --allow-run-as-root -np 1 python3 mpi_sort.py
/usr/lib/python3/dist-packages/numpy/core/fromnumeric.py:59: FutureWarning: 'DataFrame.swapaxes' is deprecated and will be removed in a future version. Please use 'DataFrame.transpose' instead.
return bound(*args, **kargs)
File akhir disalin di final_sorted_output.csv
Waktu sorting disalin di waktu_sorting.csv
Total waktu eksekusi: 51.3506 detik
# root@w-server1:~# mpirun --allow-run-as-root --hostfile hosts.txt -np 2 python3 mpi_sort.py
/usr/lib/python3/dist-packages/numpy/core/fromnumeric.py:59: FutureWarning: 'DataFrame.swapaxes' is deprecated and will be removed in a future version. Please use 'DataFrame.transpose' instead.
return bound(*args, **kargs)
File akhir disalin di final_sorted_output.csv
Waktu sorting disalin di waktu_sorting.csv
Total waktu eksekusi: 34.6221 detik
# root@w-server1:~# mpirun --allow-run-as-root --hostfile hosts.txt -np 3 python3 mpi_sort.py
/usr/lib/python3/dist-packages/numpy/core/fromnumeric.py:59: FutureWarning: 'DataFrame.swapaxes' is deprecated and will be removed in a future version. Please use 'DataFrame.transpose' instead.
return bound(*args, **kargs)
File akhir disalin di final_sorted_output.csv
Waktu sorting disalin di waktu_sorting.csv
Total waktu eksekusi: 24.7597 detik
```

Gambar 1.4 Percobaan Satu Juta Data

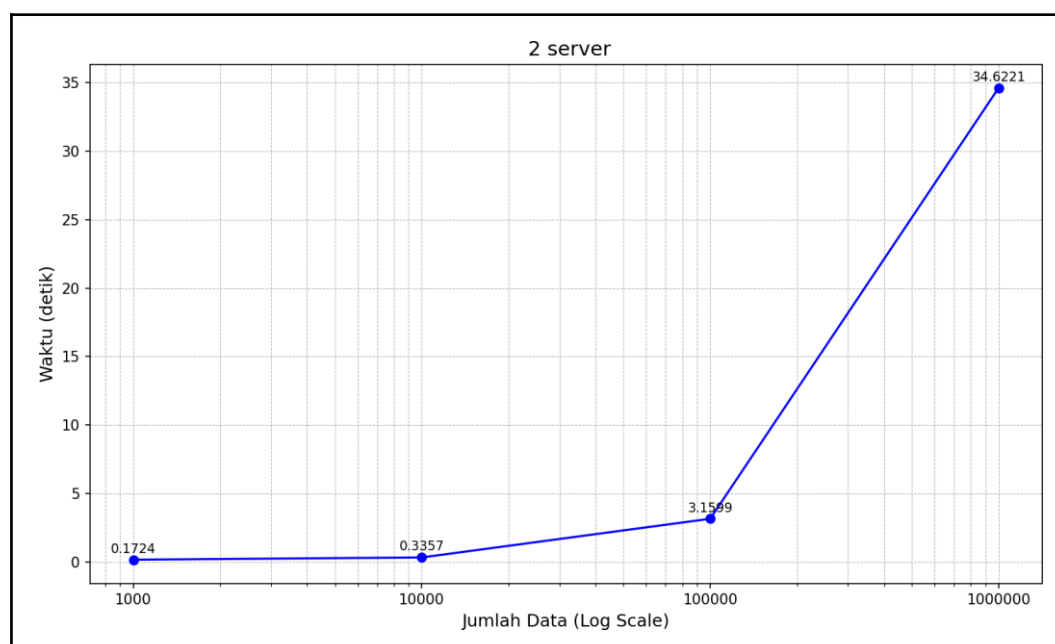
Pada gambar diatas dilakukan percobaan terhadap 1.000.000 data. Percobaan pertama dilakukan dengan 1 proses tunggal, dimana seluruh proses pengolahan data dilakukan oleh 1 CPU. Pada skenario ini, waktu eksekusi tercatat sebesar 51.3506 detik. Pada percobaan kedua, dijalankan menggunakan 2 proses paralel yang didistribusikan antar beberapa node. Hasilnya waktu eksekusi menurun menjadi 34.6221 detik. sementara pada percobaan ketiga menggunakan 3 proses paralel di dapatkan hasil eksekusi sebesar 24.7597. hasil ini menunjukkan bahwa pada data besar pemrosesan paralel dapat digunakan untuk menghemat waktu eksekusi.



Gambar 1.5 Percobaan dengan 1 Server

Grafik di atas adalah percobaan yang dilakukan dengan 1 server saja, pada jumlah data sebanyak 1000 waktu eksekusinya mencapai 0.1555 detik, untuk

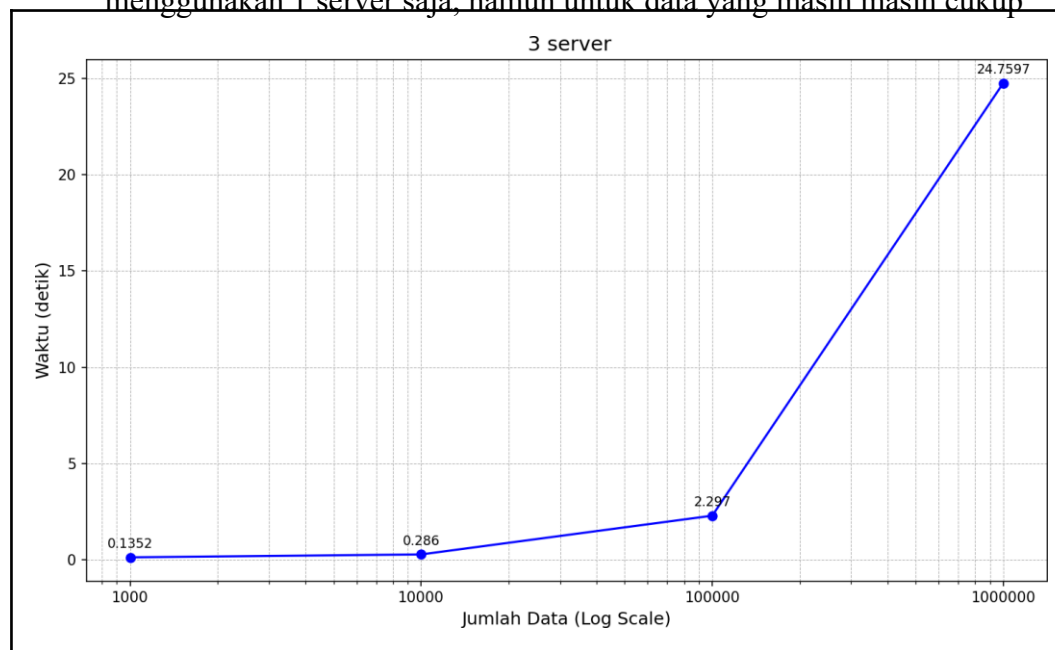
10.000 data waktu eksekusinya mencapai 0.4595, untuk data 100.000 waktu eksekusi nya 4.4191 dan untuk 1 juta data waktu eksekusi nya 51.3506 detik, untuk data kecil waktu eksekusi yang terjadi cukup cepat, namun untuk data yang besar waktu eksekusinya cukup lama jika menggunakan 1 server saja.



Gambar 1.6 Percobaan dengan 2 Server

Grafik di atas adalah percobaan yang dilakukan dengan 2 server, pada jumlah data sebanyak 1000 waktu eksekusinya mencapai 0.1724 detik, untuk 10.000 data waktu eksekusinya mencapai 0.3357 detik, untuk data 100.000 waktu eksekusi nya 3.1599 detik dan untuk 1 juta data waktu eksekusi nya 34.6331 detik, waktu

eksekusi menggunakan 2 server meningkat lebih cepat dibanding menggunakan 1 server saja, namun untuk data yang masih masih cukup



Gambar 1.7 Percobaan dengan 3 Server

Grafik di atas adalah hasil percobaan dengan 3 server. Saat jumlah data 1000, waktu eksekusi hanya 0.1352 detik. Untuk 10.000 data, waktu yang dibutuhkan adalah 0.286 detik, lalu meningkat menjadi 2.297 detik saat data mencapai 100.000, dan untuk 1 juta data, waktu eksekusinya menjadi 24.7597 detik. Dengan bertambahnya jumlah server menjadi 3, proses eksekusi menjadi lebih efisien, terutama untuk data berukuran besar.

Waktu eksekusi secara keseluruhan lebih cepat dibandingkan saat menggunakan 2 server, menunjukkan bahwa penambahan jumlah server memberikan dampak positif terhadap performa sistem paralel.

BAB V

PENUTUP

Berdasarkan serangkaian percobaan yang telah dilakukan terhadap dataset berukuran kecil (10.000 data), sedang (100.000 data), hingga besar (1 juta data), dapat disimpulkan bahwa penerapan algoritma *Merge Sort* dengan pendekatan pemrosesan paralel menggunakan MPI (Message Passing Interface) memberikan dampak positif terhadap efisiensi waktu eksekusi. Semakin besar ukuran data, semakin signifikan perbedaan waktu antara pemrosesan tunggal dan pemrosesan paralel.

Pada dataset kecil, penggunaan beberapa proses tidak menunjukkan perbedaan waktu yang sangat besar, namun tetap memberikan percepatan. Pada dataset sedang dan besar, penggunaan tiga proses secara paralel terbukti mempercepat eksekusi hingga beberapa detik. Misalnya, pada dataset 100 ribu data, waktu eksekusi turun dari 5,31 detik menjadi 4,10 detik. Sementara itu, pada dataset 1 juta data, waktu turun dari 58,26 detik menjadi 49,35 detik.

DAFTAR PUSTAKA

- Abdullah, G., Wantu, H. M., Tebay, V., Samsudin, Fathani, A. H., Rinaldi, F.,
Mardikawati, B., Anaktototy, K., Budiawan, & Gultom, J. (2024). *Transformasi pendidikan indonesia*. 02(06), 112.
- Dhani, B. M., & Pinandito, A. (2025). *Analisis Performa Worker Thread dan Cluster pada Pemrosesan Paralel Node . js*. 9(12).
- Mutaqin, F. M., Jubaedah, I., Koestianto, H., & Setiabudi, D. I. (2023). Efektif Artificial Intelligence (Ai) Dalam Belajar Dan Mengajar. *Jurnal Pendidikan: SEROJA*, 2(1).
- Rahmatullah, G. M., Fajar Zulkarnain, A., & Reza Hidayat, M. (2021). Analisis Perbandingan Performa Pemrograman Sekuensial Dan Paralel Dengan Skema Uji Matrix, Filter Dan Quick Sort. *Jurnal Teknologi Informasi Universitas Lambung Mangkurat (JTIULM)*, 6(1), 19–24. <https://doi.org/10.20527/jtiulm.v6i1.69>
- Subiyakto, B. E., Muzakkty, F., Muiz, A., & Fairuz, M. F. (2026). *MODEL PARALELISASI MENGGUNAKAN DIVIDE AND CONQUER , PIPELINE , . 3*, 951–966.
- Supriyadi, S., & Nasution, Z. (2024). Teknologi Artificial Intelligence (AI) dan Literasi Digital Mahasiswa terhadap Hasil Belajar Mata Kuliah Evaluasi Pembelajaran. *Jurnal Teknodik*, 28, 113–118. <https://doi.org/10.32550/teknodik.vi.1185>
- Waita, B. C., Yiswi, T. A., & Kristiahadi, A. (2025). Dampak Artificial Intelligence (Ai) Terhadap Pendidikan Di Indonesia. *Jurnal Pendidikan Indonesia*, 6(7), 3112–3121. <https://doi.org/10.59141/japendi.v6i7.8433>
- Wibawa, I. P. A. P., Giriantari, I. D., & Sudarma, M. (2018). Komputasi Paralel Menggunakan Model Message Passing Pada SIM RS (Sistem Informasi Manajemen Rumah Sakit). *Majalah Ilmiah Teknologi Elektro*, 17(3), 439. <https://doi.org/10.24843/mite.2018.v17i03.p20>
- Yang, A. (2022). *Approaches to the Parallelization of Merge Sort in Python*. <http://arxiv.org/abs/2211.16479>